

LibeRtAD reference manual

C++ Automatic Differentiation Engine for LibeR

Version 0.7.5

Generated 2026-07-22

15 help topics

Contents

- [ad_benchmark](#)
- [ad_benchmark_cases](#)
- [ad_checkpoint_probe](#)
- [ad_compile](#)
- [ad_engine_info](#)
- [ad_gauss_hermite](#)
- [ad_ir](#)
- [ad_load_tape](#)
- [ad_smolyak_gauss_hermite](#)
- [ad_supported](#)
- [ADModel](#)
- [libertad_gui](#)
- [libertad_workbench](#)
- [libertadWorkbenchOutput](#)
- [renderLibertadWorkbench](#)

Benchmark the persistent C++ automatic-differentiation engine

Description

Separates recording from repeated tape evaluation and compares exact CppAD derivatives with central finite differences evaluated in R. The results are intended for local regression and hardware comparison rather than as a universal performance claim.

Usage

```
ad_benchmark(  
  case = c("rosenbrock", "pk", "jacobian"),  
  iterations = 1000L,  
  warmups = 50L,  
  optimize = TRUE,  
  finite_difference = TRUE  
)
```

Arguments

case	Built-in workload returned by <code>ad_benchmark_cases()</code> .
iterations	Number of requested measured value and derivative calls. The timer may increase this count to obtain stable resolution.
warmups	Number of calls before timing.
optimize	Whether CppAD tape optimisation is enabled.
finite_difference	Include R central finite-difference comparators.

Value

An `ad_benchmark_result` containing timings, accuracy checks, engine metadata, and settings.

Examples

```
ad_benchmark_cases()  
ad_benchmark("pk", iterations = 100, warmups = 5)
```

List the built-in LibeRtAD benchmark cases

Description

List the built-in LibeRtAD benchmark cases

Usage

```
ad_benchmark_cases()
```

Value

A data frame describing the available differentiable workloads.

Benchmark nested-AD-safe CppAD checkpoint prototypes

Description

Compares repeated direct recordings with 'chkpoint_two' prototypes for an ADVAN1 interval and a 2-by-2 matrix state update. The checkpoint objects are explicitly configured for the nested AD route used by LibeRation curvature calculations. Results are diagnostic; production selection remains guarded by measured performance and tape size.

Usage

```
ad_checkpoint_probe(repetitions = 64L, evaluations = 1000L)
```

Arguments

repetitions	Number of repeated kernel uses on each outer tape.
evaluations	Number of zero-order evaluations used for timing.

Value

A named list of operation counts, accuracy checks, and timings.

Compile an R-like mathematical model

Description

Compile an R-like mathematical model

Usage

```
ad_compile(  
  code,  
  inputs = character(),  
  outputs = NULL,  
  at = NULL,  
  wrt = names(at),  
  optimize = TRUE  
)
```

Arguments

code	Character model code. The established LibeRation assignment, 'THETA(i)', 'ETA(i)', 'SIGMA(i)', and 'ERR(i)' syntax is supported.
inputs	Optional external input names. Undeclared symbols are appended as inputs in first-use order.
outputs	Optional assignment names to expose. The default exposes all assignments, with the most recent value for reassigned names.
at	Optional named recording point. When supplied, a persistent tape is recorded immediately.
wrt	Inputs with respect to which derivatives are required.
optimize	Whether CppAD should optimize the recorded tape.

Value

An 'ADModel' R6 object.

Examples

```
model <- ad_compile(  
  "CL = THETA(1) * exp(ETA(1))\nPENALTY = log(CL)^2",  
  at = c(THETA_1 = 2, ETA_1 = 0),  
  wrt = c("THETA_1", "ETA_1"),  
  outputs = "PENALTY"  
)  
model$value_gradient(c(THETA_1 = 2, ETA_1 = 0))  
model$hessian(c(THETA_1 = 2, ETA_1 = 0))
```

C++ engine and dependency information

Description

C++ engine and dependency information

Usage

```
ad_engine_info()
```

Value

A named list describing the compiled backend and dependency versions.

Examples

```
ad_engine_info()
```

Standard-normal tensor Gauss–Hermite quadrature grid

Description

Constructs a tensor-product Gaussian quadrature rule for expectations under a multivariate standard-normal distribution. The one-dimensional rule uses the Golub–Welsch eigensystem in C++; tensor nodes and log weights are also assembled in C++ to avoid repeated R-side grid construction.

Usage

```
ad_gauss_hermite(order = 5L, dimension = 1L, max_points = 100000L)
```

Arguments

order	Number of nodes per dimension.
dimension	Dimension of the normal integral. Zero returns the single empty node with unit mass.
max_points	Maximum permitted tensor-grid size. This guard prevents an accidental exponential allocation as the dimension grows.

Value

A list containing 'nodes', normalized 'weights', 'log_weights', and grid metadata.

Compile mathematical model code to a serializable intermediate representation

Description

Compile mathematical model code to a serializable intermediate representation

Usage

```
ad_ir(code, inputs = character(), outputs = NULL)
```

Arguments

code	Character model code. The established LibeRation assignment, 'THETA(i)', 'ETA(i)', 'SIGMA(i)', and 'ERR(i)' syntax is supported.
inputs	Optional external input names. Undeclared symbols are appended as inputs in first-use order.
outputs	Optional assignment names to expose. The default exposes all assignments, with the most recent value for reassigned names.

Value

A serializable 'libertad_ir' object.

Examples

```
ir <- ad_ir(  
  "CL = THETA(1) * exp(ETA(1))\nY = log(CL)",  
  outputs = "Y"  
)  
ir
```

Load a cached LibeRtAD tape

Description

Reconstructs an [ADModel] directly from a saved CppAD graph while retaining and validating the model IR and exact bundled-CppAD provenance.

Usage

```
ad_load_tape(path)
```

Arguments

path An '.rds' file created by [ADModel]\$save_tape().

Value

A ready-to-evaluate 'ADModel'.

Standard-normal Smolyak Gauss–Hermite quadrature grid

Description

Constructs an isotropic Smolyak sparse grid for multivariate standard-normal expectations. Level 'i' uses the odd-order one-dimensional Gauss–Hermite rule with $2 * i - 1$ nodes. Duplicate nodes are consolidated in C++ and negligible cancellation residues are removed.

Usage

```
ad_smolyak_gauss_hermite(level = 3L, dimension = 4L, max_points = 100000L)
```

Arguments

level	Sparse-grid accuracy level. Level one is the single origin node; increasing the level adds higher-order interactions.
dimension	Dimension of the normal integral. Zero returns the single empty node with unit mass.
max_points	Maximum permitted number of retained sparse-grid nodes.

Details

Smolyak combination weights are signed. 'weights' therefore contains the signed masses, while 'signs' and 'log_abs_weights' provide a numerically stable representation for likelihood integration.

Value

A list containing 'nodes', signed 'weights', 'signs', 'log_abs_weights', and sparse-grid metadata.

Report supported compiler constructs

Description

Report supported compiler constructs

Usage

```
ad_supported()
```

Value

A named list of accepted indexed inputs, operators, mathematical functions, conditionals, and deliberate language limitations.

Examples

```
ad_supported()
```

Pointer-backed compiled automatic-differentiation model

Description

'ADModel' is deliberately light: the serializable IR is retained for worker reconstruction, while C++ owns the executable program and persistent tape.

Public fields

`ir`

Serializable intermediate representation created by `[ad_ir()]`.

`program_ptr`

External pointer to the compiled C++ expression program.

`tape_ptr`

External pointer to the persistent CppAD tape, or 'NULL' until `[ADModel]$record()` is called.

`wrt`

Character vector naming the active differentiation inputs.

`outputs`

Character vector naming the active model outputs.

`dynamic`

Character vector naming non-differentiated inputs retained as CppAD dynamic parameters.

`dynamic_values`

Current named values of the tape's dynamic parameters.

Methods

Public methods

- `ADModel$new()`

- `ADModel$record()`
 - `ADModel$set_dynamic()`
 - `ADModel$value()`
 - `ADModel$jacobian()`
 - `ADModel$gradient()`
 - `ADModel$hessian()`
 - `ADModel$value_gradient()`
 - `ADModel$tape_info()`
 - `ADModel$tape_cache()`
 - `ADModel$save_tape()`
 - `ADModel$print()`
 - `ADModel$clone()`
-

`ADModel$new()`

Create a pointer-backed model from a validated intermediate representation.

Usage

```
ADModel$new(ir)
```

Arguments

`ir`

A 'libertad_ir' object created by `[ad_ir()]`.

Returns

A new 'ADModel' object.

`ADModel$record()`

Record a persistent CppAD tape at a named parameter point.

Usage

```
ADModel$record(at, wrt = names(at), outputs = self$outputs, optimize = TRUE)
```

Arguments

`at`

Named numeric values for every model input.

`wrt`

Input names with respect to which derivatives are required.

`outputs`

Output assignment names to place on the tape.

`optimize`

Whether to run CppAD tape optimization.

Returns

The model, invisibly.

ADModel\$set_dynamic()

Update non-differentiated inputs without recording a new tape.

Usage

```
ADModel$set_dynamic(values)
```

Arguments

`values`

Named values for every dynamic parameter. An unnamed vector may be supplied in the order shown by '\$dynamic'.

Returns

The model, invisibly.

ADModel\$value()

Evaluate model outputs with the compiled program or recorded tape.

Usage

```
ADModel$value(at, taped = !is.null(self$tape_ptr))
```

Arguments

at

Named input values. Taped evaluation requires the active 'wrt' inputs; untaped evaluation requires every IR input.

taped

Use the persistent tape when available.

Returns

A named numeric vector of model outputs.

ADModel\$jacobian()

Evaluate the exact output-by-input Jacobian of the recorded tape.

Usage

```
ADModel$jacobian(at)
```

Arguments

at

Named values for the active differentiation inputs.

Returns

A numeric matrix with one row per output and one column per input.

ADModel\$gradient()

Evaluate the exact gradient of a single-output tape.

Usage

```
ADModel$gradient(at)
```

Arguments

at

Named values for the active differentiation inputs.

Returns

A numeric gradient vector.

ADModel\$hessian()

Evaluate the exact Hessian of a single-output tape.

Usage

```
ADModel$hessian(at)
```

Arguments

at

Named values for the active differentiation inputs.

Returns

A square numeric Hessian matrix.

ADModel\$value_gradient()

Evaluate a single-output value and its exact gradient together.

Usage

```
ADModel$value_gradient(at)
```

Arguments

at

Named values for the active differentiation inputs.

Returns

A list containing 'value' and 'gradient'.

ADModel\$tape_info()

Return tape dimensions, operation counts, dynamic-parameter state, and comparison-change telemetry.

Usage

```
ADModel$tape_info()
```

Returns

A named list describing the current persistent tape.

ADModel\$tape_cache()

Create a portable cache of the optimized CppAD graph and its metadata. The serializable IR remains included as the auditable source of truth.

Usage

```
ADModel$tape_cache()
```

Returns

A 'libertad_tape_cache' list suitable for 'saveRDS()'.

ADModel\$save_tape()

Save the optimized CppAD graph for fast worker reconstruction.

Usage

```
ADModel$save_tape(path)
```

Arguments

path

Destination '.rds' path.

Returns

The normalized cache path, invisibly.

ADModel\$print()

Print a concise summary of the compiled program and tape state.

Usage

```
ADModel$print(...)
```

Arguments

...

Unused.

Returns

The model, invisibly.

ADModel\$clone()

The objects of this class are cloneable with this method.

Usage

```
ADModel$clone(deep = FALSE)
```

Arguments

deep

Whether to make a deep clone.

Launch the LibeRtAD benchmark laboratory

Description

Runs native tape microbenchmarks and, when the LibeR repository harness is available, launches the paired LibeRation/NONMEM workflow benchmark in a cancellable background process.

Usage

```
libertad_gui(  
  benchmark_root = NULL,  
  host = "127.0.0.1",  
  port = NULL,  
  launch.browser = TRUE,  
  allow_remote = FALSE  
)
```

Arguments

benchmark_root	Optional LibeR repository root or benchmark script path.
host, port, launch.browser	Passed to shiny::runApp().
allow_remote	Explicitly permit a non-loopback bind.

Value

Invisibly, the Shiny app.

LibeRtAD benchmark workbench widget

Description

Create the React/htmlwidgets LibeRtAD benchmark workbench.

Usage

```
libertad_workbench(  
  payload,  
  input_id = "libertad_workbench",  
  width = NULL,  
  height = "100vh",  
  elementId = NULL  
)
```

Arguments

payload	Workbench payload generated by <code>libertad_gui()</code> .
input_id	Shiny event prefix.
width, height	Widget dimensions.
elementId	Optional widget element id.

Value

An htmlwidget for the LibeRtAD benchmark laboratory.

Shiny output for the LibeRtAD workbench

Description

Shiny output for the LibeRtAD workbench

Usage

```
libertadWorkbenchOutput(outputId, width = "100%", height = "100vh")
```

Arguments

outputId

Output id.

width, height

CSS dimensions.

Render a LibeRtAD workbench

Description

Render a LibeRtAD workbench

Usage

```
renderLibertadWorkbench(expr, env = parent.frame(), quoted = FALSE)
```

Arguments

expr	Widget expression.
env	Evaluation environment.
quoted	Whether expression is quoted.
